

Block Shield: Secure Phishing Detection Using Machine Learning and Blockchain

1st Rohini C

*Dept. of Computer Science and
Engineering*

(Cyber Security)

Rajalakshmi Engineering College
Chennai, India

rohini.c@rajalakshmi.edu.in

2nd Keerthana S

*Dept. of Computer Science and
Engineering*

(Cyber Security)

Rajalakshmi Engineering College
Chennai, India

231901022@rajalakshmi.edu.in

3rd Madhan Kumar B

*Dept. of Computer Science and
Engineering*

(Cyber Security)

Rajalakshmi Engineering College
Chennai, India

231901028@rajalakshmi.edu.in

Abstract—Phishing blacklists react only after a page has been reported and verified, while machine learning (ML) URL classifiers can score unseen links at once but depend on the quality of their training data. Neither approach alone protects a user at the moment of the click: a list misses the first hours of a new campaign, a classifier trained on biased examples gives confident but misleading verdicts, and a locally running tool usually leaves no checkable record of what it blocked. This paper presents Block Shield, a Chrome extension backed by a local Flask service that checks every top-level navigation before the page renders. The decision rests on the URL string alone, so nothing is fetched from the destination and no visited address leaves the user's machine. Each URL passes through three stages in a fixed order: a lookup in a hash-chained ledger of previously blocked URLs, a check against an allowlist built from about 765,000 popular domains, and a Random Forest classifier over 16 lexical features taken from the URL string. The ledger stores SHA-256 digests of confirmed detections and links the entries so that later edits can be detected. It is a single-node structure inspired by blockchain, with no consensus, networking, or smart contracts. Because our first version learned the shortcut that a URL with a path looks malicious, the paths of the legitimate training URLs were rewritten before training to remove that bias. On a stratified 20% hold-out split of 201,890 URLs derived from the PhiUSIIL corpus, the classifier reached 98.12% accuracy, 99.41% precision, 96.81% recall, and a 98.09% F1-score. A full analysis takes about 30 ms at the median, and browsing continues unprotected rather than stalling if the backend is stopped. These figures need careful reading, because the split is not domain-disjoint and the paths of the legitimate URLs were generated programmatically. No external threat-intelligence service is part of the implementation. We also state the limits of the ledger, which detects casual edits but not a determined local attacker, and list the evaluation steps that remain as future work.

Index Terms—phishing detection, URL classification, random forest, hash chain, tamper-evident logging, browser extension, SHA-256

I. INTRODUCTION

A. Background

Phishing succeeds because it pushes the decision onto the user. A link arrives in a message, it looks plausible, and within seconds the person either clicks or does not. A defense that acts at the moment of that click is worth more than a warning that appears after credentials have already been typed into a fake login page.

Two kinds of URL-level defense are in common use. Reputation lists, of which browser safe-browsing services are the best known, are precise about pages that have been reported, but measurement studies show that they trail short-lived campaigns because a URL must first be reported and verified [1]. Machine learning (ML) classifiers instead score the structure of the URL itself and can react to links that nobody has reported yet [2], [3]. Their weakness is different: a classifier is only as good as its examples, and phishing datasets are easy to assemble in ways that teach the wrong lesson [4].

B. Problem Statement

We met this problem first-hand. The first version of our system was trained on synthetically generated phishing URLs and on legitimate examples that were almost all bare homepages. The model learned a shortcut that had little to do with phishing, namely that a URL with a path looks malicious. In use it flagged ordinary deep links, such as repository pages on code-hosting sites, and it missed real phishing patterns that the synthetic templates had never covered.

This experience points to three connected problems. First, reputation lists are reactive, so a newly registered phishing domain can be reachable before any list contains it. Second, an ML classifier trained on data whose classes differ for incidental reasons is confidently wrong on ordinary traffic. Third, a locally running detector normally keeps no record that can be checked afterwards, because a log of blocked pages can be edited without any trace. A practical tool must also enforce its verdict before the page renders rather than after.

C. Motivation and Research Gap

The works reviewed in Section II treat these concerns separately. Lists are strong on known threats, URL classifiers are usually reported as accuracy on a fixed corpus, and blockchain proposals in security concentrate on sharing threat intelligence among several organizations [14]. What we wanted was a tool that runs entirely on the user's machine, filters trusted domains before scoring, is trained on corrected data, and keeps an inspectable record of its own detections. Block Shield, in its present form, is the result of correcting the data problem described above and adding the missing pieces around the classifier.

D. Scope and Contributions

A Chrome extension intercepts each top-level navigation and asks a local Flask service for a verdict of Safe, Warning, or Block. The service checks a ledger of previously blocked URLs, an allowlist of popular domains, and a Random Forest over 16 URL features, in that order. Nothing is fetched from the destination and no third-party service is queried; VirusTotal and URLScan.io are not integrated. The “blockchain” component is a hash chain kept in one JSON file, with no network, consensus protocol, or smart contract. Detection is limited to the URL string, on Chromium-based browsers, for a single user on a single machine. The contributions are:

- a working pipeline, from browser navigation to a redirect issued before the page renders, that runs entirely on the user's machine;
- a training-data construction procedure built on the PhiUSIIL corpus [5] that addresses the bare-homepage bias we observed, with an account of what it does and does not fix;

- a single-node hash-chained ledger for confirmed detections, with a precise statement of which tampering it can and cannot reveal; and
- a hold-out evaluation of the classifier together with a plain account of the evaluation’s limits.

E. Security Requirements and SDG

The design aims at five requirements: enforcement before the page renders; privacy, so that visited URLs stay on the user’s machine; integrity of detection records, so that edits to the ledger can be noticed; availability, so that browsing continues when the backend is down; and few false alarms on well-known sites. By protecting users against credential theft, the work contributes to safer digital infrastructure and therefore relates to UN Sustainable Development Goal 9 (Industry, Innovation and Infrastructure).

II. RELATED WORKS

A. Existing Approaches and Previous Research

Blacklists remain the default browser defense. Sheng et al. measured how quickly phishing sites appeared on such lists and found weak coverage at the start of a campaign, since listing depends on a report and a verification step [1]. Lists are still the right tool for known threats. The first stretch of a campaign is the gap that URL classifiers try to fill. Garera et al. showed that URL structure alone, including brand names inside unrelated hosts and unusual path patterns, carries useful signal [6]. Ma et al. cast detection as supervised learning over lexical and host-based features [2], and Blum et al. combined lexical features with online learning so that a model could follow changing URL patterns [7]. Whittaker et al. described an automated page classifier operating at large scale that feeds a blacklist [8]. Sahingoz et al. compared several classifiers on URL-derived features and reported Random Forest as the strongest on their feature set [9]. Deep models such as URLNet learn from characters and words directly instead of using handcrafted features [10], and the survey by Sahoo et al. maps the wider field [3]. Block Shield belongs to the handcrafted-feature, tree-ensemble part of this space; we did not compare it with deep models.

B. Data and Evaluation Pitfalls

Arp et al. review recurring mistakes in security-oriented ML, among them sampling bias, spurious correlations, and evaluation that does not reflect deployment [4]. The failure of our first version is a small instance of the spurious-correlation problem. PhiUSIIL publishes a large labeled corpus of phishing and legitimate URLs together with page-derived features [5]; we use only its URL strings.

C. Tamper-Evident Records and Blockchain

Linking records by hashes so that their history can be checked goes back to the time-stamping scheme of Haber and Stornetta [11]. Schneier and Kelsey applied chained hashes together with keys to audit logs, so that entries written before a compromise could not be quietly rewritten [12]. The Bitcoin blockchain added distributed consensus about who may append [13]. In security work, blockchains have mostly been proposed for sharing threat intelligence between organizations, for example BLOCIS [14]. Those designs need several cooperating parties.

TABLE I
COMPARISON OF EXISTING APPROACHES AND BLOCK SHIELD

Approach	Refs.	Strength	Limitation
Reputation lists	[1]	Precise on reported pages	Late at the start of a campaign
Lexical ML classifiers	[2], [6], [7], [9]	Scores unseen URLs at once	Quality depends on training data [4]
Blockchain threat sharing	[14]	Shares intelligence across parties	Needs several cooperating parties
Hash-chained audit logs	[11], [12]	Past entries cannot be quietly rewritten	Record keeping, not detection
Block Shield	this work	Ledger, allowlist, and Random Forest run locally before render	URL-only; single-node ledger

D. Comparison and Research Gap

Taken together, the reviewed work leaves three things open for a tool of this kind. Lists alone are late, URL classifiers depend on how their data was built, and the blockchain-based security designs we reviewed assume several cooperating parties, which a single-user browser tool does not have. Block Shield therefore places a trusted-domain filter in front of a classifier trained on a corrected corpus, keeps a local hash-chained record of confirmed detections, and enforces the verdict inside the browser before the page renders. Because the chain serves local integrity checking only, it sits nearer to the audit-log tradition [11], [12] than to distributed threat sharing, which is why we call it blockchain-inspired rather than a blockchain.

III. PROPOSED METHODOLOGY

A. Overview and Design Choice

Block Shield has a browser half and a server half. The browser half is a Manifest V3 extension [15] whose service worker listens for top-level navigations and enforces the verdict. The server half is a Flask application on the same machine that owns the model, the allowlist, the ledger, and the database. One design choice shapes everything else: the decision uses the URL string and nothing else. The system never requests the destination page, so it is not exposed to page content and adds no traffic toward a suspicious host. The price is that anything visible only inside the page, such as a cloned login form on an innocuous-looking address, is out of reach.

B. URL Feature Extraction

All 16 features are computed from the URL string and its parsed parts, and the `tldeextract` library separates subdomain, registrable domain, and suffix. Six are counts: `url_length`, `num_dots`, `num_hyphens`, `num_digits`, `num_slashes`, and `num_query_params` (the number of “=” plus “&”). Seven are binary flags: `has_ip` (raw IPv4 host), `has_at_symbol`, `has_https`, `has_suspicious_words` (a lure word such as login, verify, or secure), `has_brand_impersonation` (a listed brand appears in the URL but is not the registrable domain), `is_shortener`, and `has_double_slash_redirect`. The remaining three are structural or reputation signals: `subdomain_count`, `domain_length`, and `is_popular_domain` (the registrable domain is on the allowlist and not in the exclusion list). The same function produces the features during training and during live scoring, which rules out skew between the two. The features are cheap, since each is a string operation or a set lookup, but they

are also proxies. Slash, digit, and length counts stand in for path complexity, and a phishing kit that adopts ordinary-looking URL shapes can move them.

C. ML-Based Phishing Detection

The classifier is a Random Forest [17], [18]. Tree ensembles suit this problem for practical reasons: they handle a mix of counts and binary flags without scaling, capture interactions between features, cost little at inference time, and expose feature importances that a security tool benefits from being able to show. We make no claim that Random Forest beats other algorithms here, since no comparative experiment was run. At run time the service loads the serialized model once. For each URL that reaches the classifier, the forest returns the mean of the per-tree phishing probabilities,

$$p(u) = \frac{1}{150} \sum_{t=1}^{150} p_t(x(u)) \quad (1)$$

where $x(u)$ is the 16-feature vector of URL u and p_t is the probability output by tree t . This value is the `ml_score` returned to the extension. Training is an offline script run by hand; the deployed service never retrains itself.

D. Hash-Chained Ledger

URL hashing. Before hashing, a URL is stripped of surrounding whitespace and lowercased. SHA-256 [19] over the result gives a 64-character hexadecimal digest, and this digest, not the URL text, is what the ledger stores and compares. Digests are not a privacy guarantee: URLs are low-entropy, so digests of guessable addresses can be checked by dictionary, and the plain URL is in any case kept in the scan history. Matching is exact. The http and https forms of an address, a trailing slash, or an extra query parameter each yield a different digest, so a previously blocked page that reappears under a slightly different string falls through to the classifier.

Chain structure and verification. Each block holds an index i , the URL digest d_i , the previous block's hash, a timestamp t_i , and its own hash, computed over the other fields:

$$h_i = \text{SHA-256}(i || d_i || h_{i-1} || t_i). \quad (2)$$

A genesis block with all-zero hashes is created the first time the file is accessed, and a fresh installation starts with an empty ledger. Only Block verdicts produced by the classifier are appended; Warnings are not. Writes are guarded by a lock and performed by writing a temporary file and renaming it over the original, so a crash does not leave a half-written chain. Lookup scans the chain linearly for a matching digest. Verification is also one linear pass: it recomputes every block's hash and checks that each stored previous-hash equals the hash of the block before it.

It helps to be exact about what this detects. If someone alters a stored digest or timestamp without recomputing the hash, the block no longer matches its own hash. If they also recompute that hash, the next block's stored previous-hash no longer matches. A casual edit is therefore found. But nothing in the file is secret or signed, and the head of the chain is not recorded anywhere else, so a person with write access who recomputes every later hash can produce a chain that passes verification. The ledger gives evidence against accidental corruption and unsophisticated edits; it does not protect against a determined local attacker. There is no smart contract: no contract language, chain, wallet, or Web3 library. Append, membership query, and integrity check are ordinary Python functions in one module, and only the `/scan` handler appends.

E. Allowlist and Decision Engine

For every navigation the service applies its checks in a fixed order and stops at the first one that yields a verdict. The order reflects cost and confidence. A previous confirmed detection is the strongest evidence the system holds and is cheap to look up, so the ledger goes first. The allowlist goes second because it removes the most common case, legitimate traffic to well-known sites, before any scoring. The classifier is last and is the only stage that returns a graded score. The rules are as follows.

- URLs with a browser-internal scheme (`chrome://`, `about:`, `edge://`, `brave://`) are Safe and skipped.
- If the URL digest is in the ledger, the verdict is Block (source: ledger).
- If the registrable domain is on the allowlist and not excluded, the verdict is Safe (source: allowlist).
- Otherwise the score decides: Block for `ml_score` ≥ 0.60 , Warning for $0.30 \leq \text{ml_score} < 0.60$, and Safe below 0.30.

There is no fusion of several scores; only the classifier output ever reaches a threshold. The two thresholds are read from environment variables and default to these values. They were set by hand as practical operating points and were not tuned on validation data. The allowlist excludes URL shorteners and free-hosting platforms, because a trusted registrable domain there says little about the page behind a given link, but it still cannot catch abuse such as an open redirect hosted on a popular domain. When the classifier does not run, the history records a score of 0.0. In terms of cost, ledger lookup grows linearly with the number of blocked URLs, the allowlist test is a constant-time set lookup, and classification costs the same for every URL that reaches it.

F. Browser Enforcement, Backend, and Storage

The extension declares the `webNavigation`, `tabs`, `storage`, and `activeTab` permissions and host access to HTTP and HTTPS pages plus the local service. It has no content scripts. Its service worker acts only on top-level frames, so iframes are ignored. It first checks a five-minute in-memory cache and otherwise posts the URL to the backend. The verdict is written to `chrome.storage.local` for the popup, and the toolbar badge is coloured by verdict. A Warning produces a desktop notification and lets the page load. A Block produces a notification and replaces the tab's target with `blocked.html`, which receives the URL, score, and source as query parameters and offers "Take me back" and "Proceed anyway", so the final choice stays with the user. If the backend cannot be reached, the fetch error is caught and the verdict becomes "Unknown", meaning the browser keeps working without protection instead of stalling.

The Flask service exposes four endpoints: `/scan` (POST, returns verdict, score, source, and latency), `/history`, `/health` (model status, ledger length and validity, database type), and `/ledger/verify`. `/scan` returns HTTP 400 when the URL is missing. Every scan is written to one `scan_history` table (id, url, mlscore, verdict, scannedat), in SQLite by default or MySQL 8.0 when configured, using parameterized queries and only INSERT and SELECT statements. The ledger is deliberately separate: a JSON file holding digests only, with no link to the table.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the complete flow of one navigation. The Chrome extension's service worker intercepts the top-level navigation and posts the URL to the Flask backend on localhost, where the `scan()` decision engine first checks the hash-chained ledger, and a match

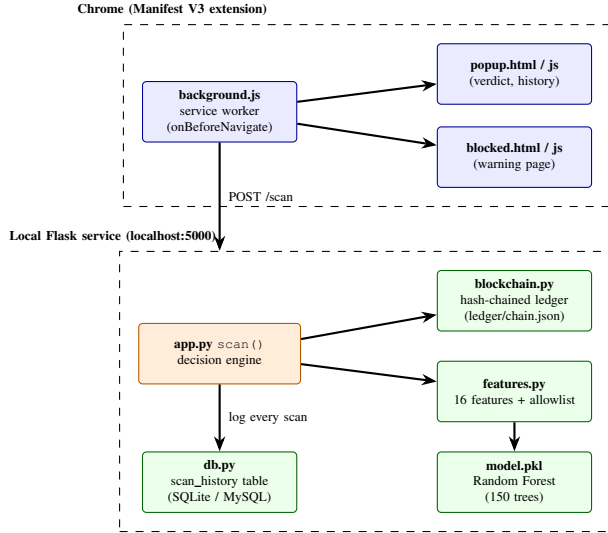


Fig. 1. Block Shield architecture.

returns Block immediately. Otherwise the URL is checked against the trusted-domain allowlist, which returns Safe for popular domains, and only the remaining URLs go through feature extraction, where the 16 features are computed, and into the Random Forest model, which outputs a phishing probability. The decision engine compares this probability with the Warning and Block thresholds to assign Safe, Warning, or Block, and a Block from the classifier is also appended to the ledger. Each scan is logged in the scan-history database, and the verdict returns to the extension, which responds in the browser by allowing the page, raising a notification, or redirecting the tab to the warning page.

V. EXPERIMENTAL SETUP

A. Dataset

The phishing and legitimate URL lists come from the PhiUSIIL corpus [5], as stated in the project’s data-preparation code and README; we did not independently re-verify the provenance of the files. The allowlist is a separate artifact: a static, gzip-compressed list of 765,457 popular domains sampled once from the Chrome UX Report [16] and bundled with the project. It is not refreshed at run time.

The raw legitimate list consisted almost entirely of bare homepages with no path or query string, while many phishing URLs carried long paths. A classifier trained on that combination can separate the classes by asking whether a path exists, which is exactly the shortcut our first version learned. The script `generate_dataset.py` therefore rewrites the path and query of roughly 70 to 90 percent of the legitimate URLs using randomly generated templates (lowercase slugs, numeric identifiers, and query-parameter patterns) with the random seed fixed at 42. Domains are left untouched, the phishing list is used as provided, and the two classes are balanced (Table II). The domains of both classes are therefore real, but the legitimate paths are synthetic, so the corpus cannot show how the model treats genuine legitimate deep links.

B. Data Preprocessing and Feature Extraction

Each URL is parsed with `tlldextract`, and the 16 features of Section III-B are computed from the string and its parsed components. No scaling or normalization is applied, because tree-based models do

TABLE II
DATASET AND SPLIT STATISTICS

Item	Count
Phishing URLs in source file (lines)	100,944
Legitimate URLs in source file (lines)	134,849
Balanced training CSV (examples)	201,890
Training set (80%)	161,512
Test set (20%)	40,378
Test examples per class	20,189 / 20,189
Allowlist domains	765,457

not need it. The same extraction function is used for training and for live scoring.

C. Training and Testing

The dataset is split 80/20 with stratification on the label and a random state of 42, giving 161,512 training and 40,378 test URLs. Metrics are computed with phishing as the positive class: accuracy, precision, recall, and F1-score, together with the confusion matrix. The training script `train_model.py` writes the model and a `metrics.json` file, and the reported results are taken from that file.

D. Random Forest Configuration

The classifier is scikit-learn’s `RandomForestClassifier` with 150 trees, maximum depth 20, a minimum of 5 samples per leaf, balanced class weights, and a random state of 42. These values were chosen by hand and were not searched.

E. Experimental Environment

The software environment follows the pinned requirements: Flask 3.0.3, Flask-CORS 4.0.1, scikit-learn 1.5.1, pandas 2.2.2, NumPy 1.26.4, joblib 1.4.2, tldextract 5.1.2, and mysql-connector-python 9.0.0. Tests ran on a machine with an AMD Ryzen 5 8645HS processor, 16 GB RAM, and 64-bit Windows 11, using Python 3.12.3 and Google Chrome 153.0.8010.54. To reproduce the pipeline, `generate_dataset.py` builds the CSV, `train_model.py` fits the model, `app.py` starts the service, and the extension is loaded unpacked in Chrome’s developer mode. Cross-validation, ROC-AUC or calibration analysis, a threshold sweep, hyperparameter search, comparison with other classifiers, and evaluation on data outside the training distribution were not carried out.

VI. RESULTS AND DISCUSSION

A. Classification Performance

On the 40,378-URL hold-out set the classifier reached 98.12% accuracy, 99.41% precision, 96.81% recall, and a 98.09% F1-score (Fig. 2). The confusion matrix (Fig. 2) shows 20,073 legitimate URLs classified correctly and 116 flagged as phishing, and 19,545 phishing URLs caught and 644 missed. From these counts we derived a false-positive rate of 0.57% (116 of 20,189) and a false-negative rate of 3.19% (644 of 20,189).

The model errs in an asymmetric way. About 1 phishing URL in 31 was missed, while only 116 legitimate URLs were flagged. For a blocker that interrupts browsing, a low false-positive rate is the more visible virtue, but the missed phishing URLs are the costlier errors. In deployment the allowlist removes many legitimate URLs before the classifier sees them, so the false-positive rate a user experiences on popular sites should be lower than 0.57%. We would expect that, but we did not measure it.

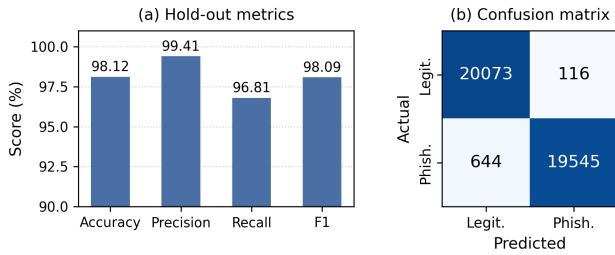


Fig. 2. Hold-out results: (a) accuracy, precision, recall, and F1-score; (b) confusion matrix (phishing is the positive class).

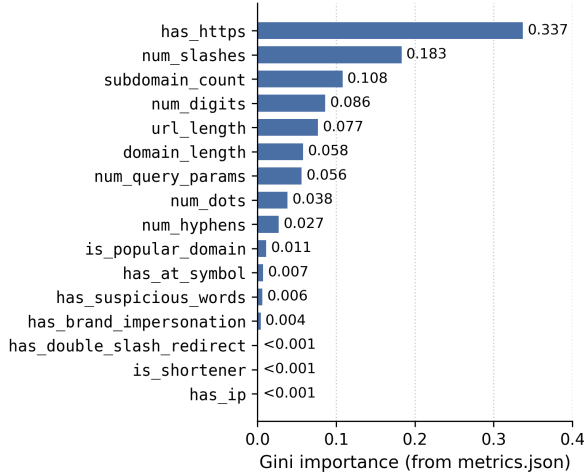


Fig. 3. Gini feature importances of the trained Random Forest, from the project’s metrics file.

B. Feature Importance

Fig. 3 shows the importances recorded after training. Three features, `has_https`, `num_slashes`, and `subdomain_count`, carry 62.8% of the total, and the top five carry 79.1%. The features `has_ip`, `is_shortener`, and `has_double_slash_redirect` contribute almost nothing. This most likely reflects how rarely those patterns occur in this corpus rather than any lack of relevance in the wild, but the data here cannot settle the question. The popularity flag has a low importance (1.1%), partly because the allowlist handles such domains before the classifier is reached.

C. System Behavior, Latency, and Interface

For the README smoke-test address `http://secure-paypal-login.verify-account.top/login`, which is not on the allowlist and not in a fresh ledger, a real run returned an `ml_score` of 0.98, well above the Block threshold. The functional checks we ran behaved as the decision logic predicts: browser-internal pages were skipped as Safe, a second visit to a blocked URL was answered from the ledger, a mid-range score produced a Warning while the page loaded, editing one byte of `chain.json` made `/ledger/verify` return false, stopping the backend gave an Unknown verdict with browsing continuing, and a POST to `/scan` without a URL returned HTTP 400. Table III gives the response latency recorded by the service over 30 requests per decision path. Figs. 4 and 5 show the interface and the backend.

TABLE III
BACKEND LATENCY PER DECISION PATH (MS)

Path	Median	95th pct.	Requests
Ledger hit	0.00	2.01	30
Allowlist hit	7.12	14.95	30
Full analysis	29.54	83.25	30

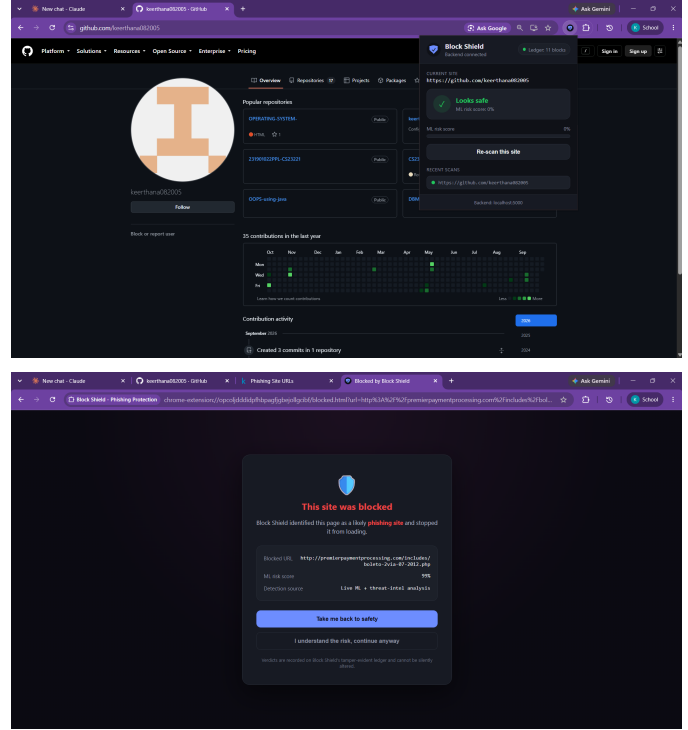


Fig. 4. Block Shield interface: (top) Safe verdict and scan details for a safe URL; (bottom) full-page warning shown after a Block.

D. Discussion

What the numbers support. The hold-out scores show that the 16 features carry enough signal to separate this corpus. They do not show that the same performance holds on unseen phishing campaigns, for three reasons. The split is random at the row level, so domains and even near-identical URLs can appear in both training and test data. The legitimate paths are synthetic, so part of what the model learns may be the fingerprint of our generator. And the most important feature is `has_https`: a model that has learned that plain HTTP means phishing will do well where that is true and poorly against phishing served over HTTPS, which is cheap to obtain. A domain-disjoint split, an ablation without `has_https`, and a test on recent live URLs would each be informative.

Ledger and classifier together. The classifier is probabilistic, and its answer for a URL can change if the model is retrained or the thresholds move. The ledger fixes the outcome: once an exact URL has been blocked, later visits are blocked cheaply and deterministically. The same property is a weakness, because the code has no way to remove an entry, so a false positive becomes permanent for that exact URL and “Proceed anyway” does not change the next visit. The ledger contributes tamper evidence for a local record and a fast path for repeat detections; it contributes nothing to detection accuracy and does not provide decentralization or immutability.

```

db_type      : sqlite
ledger_length : 12
ledger_valid  : True
model_error   :
model_loaded  : True
status        : ok

(venv) PS C:\Users\keert\Downloads\BlockShield (1)\BlockShield\backend> Invoke-Res
>> -Method POST
>> -ContentType "application/json"
>> -Body '{"url":"http://secure-paypal-login.verify-account.top/login"}'

latency_ms : 2.01
message    : Already reported as phishing
source     : ledger
url        : http://secure-paypal-login.verify-account.top/login
verdict    : Block

```

Fig. 5. Backend/API output.

Trade-offs. The allowlist removes the class of false alarms that our first version suffered from, at the cost of trusting every non-excluded page on a popular domain, and its snapshot ages because it is not refreshed. Local operation means no visited URL is sent to a third party by Block Shield’s own code, which is a real privacy advantage, but it also means the system learns nothing from other users. Failing open keeps browsing usable, yet anyone able to stop the local service also switches the protection off. Finally, correcting the training data mattered more than any modeling choice we made; we know this from the project’s change history and from qualitative behavior, not from a controlled comparison.

VII. CONCLUSION AND FUTURE WORK

A. Conclusion

Block Shield combines a hash-chained ledger of past detections, a popularity allowlist, and a Random Forest over 16 URL features, and enforces the result from a Chrome extension before a page renders. On a 20% hold-out split of 201,890 URLs the classifier reached 98.12% accuracy with 99.41% precision and 96.81% recall. We treat this as evidence that the features are informative on this corpus, not as a measure of real-world performance, because the split is not domain-disjoint and the legitimate paths are synthetic. The ledger gives tamper evidence against casual edits rather than the guarantees of a distributed blockchain. The design depends on no external service, which keeps it private and simple.

B. Future Work

None of the following is implemented. On the evaluation side, the priorities are a domain-disjoint split, cross-validation, ROC and calibration analysis, a threshold study, a comparison with at least a linear model and a gradient-boosted ensemble, and a test on recently reported live phishing URLs. Replacing the synthetic legitimate paths with observed ones would remove the largest doubt about the corpus. For the ledger, normalizing URLs before hashing would close much of the exact-match gap, anchoring the chain head with a signature or external timestamp would raise the cost of undetected tampering, and a removal or expiry mechanism should be designed together with that. A distributed ledger would only make sense if detections were shared between installations. Reputation or sandbox services such as VirusTotal or URLScan.io could add independent evidence for URLs in the Warning band, at the price of sending URLs to a third party; until then, interface strings that mention threat-intelligence cross-verification should be corrected. The prototype also needs hardening before real use: authentication and restricted origins for the local

API, localhost-only binding with debug mode off, escaped rendering of URL text in the popup, and an automated test suite. Support for other browsers and page-content or visual-similarity signals would extend detection beyond the URL string.

REFERENCES

- [1] S. Sheng, B. Wardman, G. Warner, L. F. Cranor, J. Hong, and C. Zhang, “An empirical analysis of phishing blacklists,” in *Proc. 6th Conf. Email and Anti-Spam (CEAS)*, Mountain View, CA, USA, 2009.
- [2] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: Learning to detect malicious web sites from suspicious URLs,” in *Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, Paris, France, 2009, pp. 1245–1254.
- [3] D. Sahoo, C. Liu, and S. C. H. Hoi, “Malicious URL detection using machine learning: A survey,” arXiv:1701.07179, 2017.
- [4] D. Arp et al., “Dos and don’ts of machine learning in computer security,” in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.
- [5] A. Prasad and S. Chandra, “PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning,” *Computers & Security*, vol. 136, Art. no. 103545, 2024.
- [6] S. Garera, N. Provos, M. Chew, and D. A. Rubin, “A framework for detection and measurement of phishing attacks,” in *Proc. ACM Workshop on Recurring Malcode (WORM)*, Alexandria, VA, USA, 2007, pp. 1–8.
- [7] A. Blum, B. Wardman, T. Solorio, and G. Warner, “Lexical feature based phishing URL detection using online learning,” in *Proc. 3rd ACM Workshop on Artificial Intelligence and Security (AISec)*, Chicago, IL, USA, 2010, pp. 54–60.
- [8] C. Whittaker, B. Ryner, and M. Nazif, “Large-scale automatic classification of phishing pages,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2010.
- [9] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
- [10] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, “URLNet: Learning a URL representation with deep learning for malicious URL detection,” arXiv:1802.03162, 2018.
- [11] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” *J. Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.
- [12] B. Schneier and B. Kelsey, “Secure audit logs to support computer forensics,” *ACM Trans. Information and System Security*, vol. 2, no. 2, pp. 159–176, 1999.
- [13] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [14] S. Gong and C. Lee, “BLOCIS: Blockchain-based cyber threat intelligence sharing framework for sybil-resistance,” *Electronics*, vol. 9, no. 3, Art. no. 521, 2020.
- [15] Google, “Chrome extensions documentation (Manifest V3),” Chrome for Developers. [Online]. Available: <https://developer.chrome.com/docs/extensions>. September 29, 2026.
- [16] Google, “Chrome UX Report (CrUX),” Chrome for Developers. [Online]. Available: <https://developer.chrome.com/docs/crux>. September 29, 2026.
- [17] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [18] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [19] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.